设计模式常见面试题

为什么要学习设计模式?

- 看懂源代码：如果你不懂设计模式去看Jdk、Spring、SpringMVC、IO等等等等的源码，你会很迷茫，你会寸步难行
- 看看前辈的代码：你去个公司难道都是新项目让你接手？很有可能是接盘的，前辈的开发难道不用设计模式?
- 编写自己的理想中的好代码：我个人反正是这样的，对于我自己开发的项目我会很认真，我对他比对我女朋友还好，把项目当成自己的儿子一样

设计模式有哪几类?

- 创建型
  - 单例模式
  - 工厂方法
  - 抽象工厂
  - 建造者
  - 原型
- 结构性
  - 适配器
  - 代理
  - 装饰
  - 组合
  - 桥接
  - 外观
  - 享元
- 行为型
  - 访问者
  - 模板模式
  - 策略模式
  - 观察者模式
  - 责任链
  - 等等。。。

设计模式7大设计原则?

- 开放封闭原则
- 里氏替换原则
- 依赖导致原则
- 接口隔离原则
- 迪米特法则
- 单一职责原则

咕泡出品 必属精品

在工作中你用过哪些设计模式？或者熟悉哪些设计模式？

我用过单例，工厂，代理，适配器，原型，建造者，策略设计模式。

追问：你说下单例模式?

单例模式是为了保证对象在jvm中对象的唯一，java对于单例模式提供了几种解决方案

第一种：懒汉式

懒汉式分为双重检查锁和静态内部类

第二种：饿汉式

饿汉式分为静态成员变量和静态代码块

第三种：注册式

枚举：官方推荐

容器：Spring中的单例就是容器式单例

第四种：ThreadLocal

在单个线程中单例

追问：哪些方式可以破坏单例，如果解决?

第一种：序列化：除了枚举的单例不会被序列化破坏，其他的都能。可以去添加readResolve方法，jdk底层会判断是否有该方法，然后该方法返回原来的单例对象，这个缺点是还是会生成一个新的对象，只是不会返回。会导致内存浪费。

第二种：反射：可以在构造函数中去抛异常

第三种：克隆模式：都已经是单例了，就不要去使用克隆模式了。

追问：你说下代理模式，代理分为几种?

静态代理和动态代理

追问：动态代理想要得到代理类需要哪些参数?

jdk：类加载器，接口，handler，实现被代理类实现的接口

cglib: 会继承我们的被代理类

追问：jdk代理和cglib的区别是什么?

JDK 是字节码生成快，cglib 是字节码生成慢，jdk就生成一个文件，cglib生成三个

一个实现接口，一个继承类

jdk调用慢，cglib调用快，jdk通过反射，cglib通过索引定位到方法，jdk1.8后，反射的性能很大的提高

追问：代理类可以再被代理吗?

1.jdk代理类被jdk代理: 只有一种情况下会可以，使用不用的handler的时候。其他情况不行：JDK代理后再进行一层jdk代理，循环调用，造成代理的死循环

咕泡出品 必属精品

2.jdk代理类被cglib代理：不可以 。因为jdk代理后的类是public final类型的，cglib不能对final的类进行代理

3.cglib代理类被jdk代理：不可以。因为CGlib代理后已经改变了类的签名，而jdk必须知道类的接口和实现

4.cglib代理类被cglib代理：不可以。测试结果不可以 Caused by: java.lang.ClassFormatError: Duplicate method name&signature

Spring常见面试题

1.spring的模块有哪些?

Spring Core：基础，可以说Spring其他所有的功能都依赖于该类库。主要提供IOC和DI功能。

Spring AOP：提供面向切面的编程实现。

Spring JDBC：Java数据库连接。

Spring JMS：Java消息服务。

Spring ORM：用于支持Hibernate等ORM工具。

Spring Web：为创建Web应用程序提供支持。

Spring Test：提供了对JUnit和TestNG测试的支持。

spring tx:事务

2.说下你对spring的理解?

1.Spring框架帮我们管理对象及其依赖关系

2.是基于POJO的轻量级和最小侵入性编程

3.通过依赖注入进行控制反转

4.通过依赖注入和面向接口来松耦合

3.对spring ioc的理解和使用?

以前没有spring的时候，我们需要得到一个对象，都是自己主动去new一个对象，然后通过set方法给对象注入属性，但是这种动作其实是一个重复的动作，所以spring提供ioc的容器解决方案，在容器启动的时候就把许多需要实例化和属性注入的bean都提前做好并放入到一个map中存储起来。这就是控制反转，原来的控制全在用户，现在的控制权完全交给了容器，在bean实例化后，通过反射对属性进行依赖注入

有两种使用方式，一种是xml的方式，一种是注解的方式。

xml的加载方式，首先在spring的xml中通过bean标签配置我们需要注入的bean,当扫描到所有的bean后，首先把bean包装成BeanDefinition，放入到list中，然后循环这个list去创建bean,创建bean的步骤为实例化->属性注入->初始化->aop,然后最终放入到spring的一级缓存中保存起来。

注解方式是通过在类上面加上spring的注解去扫描，比如controller，service等，后续步骤和xml基本一样。

4.Bean是如何创建的?

咕泡出品 必属精品

第一步：找到哪些bean需要去实例化；xml方式去找，一种注解的方式去找，封装成一个BeanDefinition放入到list中

第二步：循环list,创建一个个bean对象

1.实例化

2.属性注入

3.初始化

4.aop

第三步：放入到一级缓存中

5.spring注入bean的方式有哪些

1.通过@Configuration+@Bean的方式注入

2.通过实现ImportBeanDefinitionRegistrar接口可以往容器中注入BeanDefinition，从而注入bean

3.通过实现ImportSelector接口可以往spring容器中批量注入Bean

4.通过实现FactoryBean接口可以往Spring容器中自定义Bean

5.@Componet + @ComponentScan

6.Bean的生命周期?

spring的bean的生命周期主要是创建bean的过程，一个bean的生命周期主要是4个步骤，实例化，属性注入，初始化，销毁，但是对于一些复杂的bean的创建，spring会在bean的生命周期中开放很多的接口，可以让你加载bean的时候对bean做一些改变，因此spring的bean的生命周期总共有以下几步：

首先在spring中有一些特殊的bean会介入到其他bean的声明周期当中去，所以一个普通的bean的生命周期为：

1. 实现了BeanFactoryPostProcessor接口的bean,在加载其他的bean的时候，也会调用这个bean的postProcessBeanFactory方法，可以在这个步骤去对bean中的属性去赋值。设置年龄初始化18等等。
2. 实现了InstantiationAwareBeanPostProcessor接口的bean，会在实例化bean之前调用postProcessBeforeInstantiation方法
3. 然后在对bean进行实例化
4. 对bean进行属性注入
5. 对bean进行初始化，在初始化中，包含了以下几个步骤：

   1）实现了BeanFactoryAware接口，会先调用setBeanFactory方法

   2）实现了BeanNameAware接口，会先调用setBeanName方法

   3）实现了BeanPostProcessor接口，会先调用postProcessBeforeInitialization方法

   3）实现了InitializingBean接口，会调用afterPropertiesSet方法

   4）然后在进行aop后置处理，通过实现BeanPostProcessor接口,在postProcessAfterInitialization方法中进行动态代理
6. 销毁

当然还有一些其他的步骤，在此就不一一列举了，bean的生命周期总共有18步。

咕泡出品 必属精品

7.Spring常用的设计模式有哪些?

1.工厂设计模式：Spring使用工厂模式可以通过BeanFactory或ApplicationContext创建bean对象。

2.单例设计模式：Spring中bean的默认作用域就是singleton。spring的一级缓存就是使用的容器式单例

3.代理设计模式：Spring AOP就是基于动态代理的。如果要代理的对象，实现了某个接口，那么Spring AOP会使用JDK Proxy，去创建代理对象，而对于没有实现接口的对象，就无法使用JDK Proxy去进行代理了，这时候Spring AOP会使用Cglib，这时候Spring AOP会使用Cglib生成一个被代理对象的子类来作为代理。

4.模板方法设计模式：Spring中jdbcTemplate、hibernateTemplate等以Template结尾的对数据库操作的类，它们就使用到模板模式。

5.观察者设计模式：Spring事件驱动模型就是观察者模式很经典的应用。

spring的事件流程：

1）定义一个事件: 实现一个继承自 ApplicationEvent，并且写相应的构造函数

2）定义一个事件监听者：实现 ApplicationListener 接口，重写 onApplicationEvent() 方法

3）使用事件发布者发布消息: 可以通过 ApplicationEventPublisher 的 publishEvent() 方法发布消息

6.适配器设计模式：适配器模式使得接口不兼容的那些类可以一起工作，其别名为包装器

在Spring MVC中，DispatcherServlet根据请求信息调用HandlerMapping，解析请求对应的Handler，解析到对应的Handler（也就是我们常说的Controller控制器）后，开始由HandlerAdapter适配器处理

7.装饰者设计模式：装饰者设计模式可以动态地给对象增加些额外的属性或行为。相比于使用继承，装饰者模式更加灵活

Spring 中配置DataSource的时候，DataSource可能是不同的数据库和数据源。我们能否根据客户的需求在少修改原有类的代码下切换不同的数据源？这个时候据需要用到装饰者模式。

8.策略设计模式：Spring 框架的资源访问接口就是基于策略设计模式实现的

8.BeanFactory和ApplicationContext

功能	BeanFactory	ApplicationContext
Bean实例化	yes	yes
集成生命周期管理	no	yes
BeanPostProcessor自动注册	no	yes
BeanFacoryPostProcessor自动注册	no	yes
便捷的MessageSoure获取（国际化）	no	yes
内建ApplicationEvent发布机制	no	yes

因为`ApplicationContext`包含`BeanFactory`的所有功能，所以通常建议在普通BeanFactory中使用。

BeanFactory ： 延迟注入,相比于ApplicationContext 来说会占用更少的内存，程序启动速度更快。

ApplicationContext： 容器启动的时候，不管你用没用到，一次性创建所有 bean 。BeanFactory 仅提供了最基本的依赖注入支持，ApplicationContext 扩展了 BeanFactory ,除了有BeanFactory的功能还有额外更多功能，所以一般开发人员使ApplicationContext会更多

9.spring是如何解决循环依赖的?

1.构造函数循环依赖（无法解决）

> 首先要解决循环依赖就是要先实例化，然后放入三级缓存暴露出来，那么如果是构造函数这一步循环依赖，实例化的时候就会产生无限递归创建，所以不能别解决
>
> 如果是延迟加载的话可以解决(另当别论)

2.setter方式的多例的循环依赖（无法解决）

> 如果是多例的，在容器初始化的时候，不会去创建，所以早期没有放入到三级缓存中暴露出来，所以无法解决循环依赖，会报错

3.setter方式的单例循环依赖（A依赖B，B依赖A）

> A与B循环依赖，都要被代理的情况
>
> 1.A实例化->A放入三级缓存->依赖注入B->B实例化->B放入三级缓存->依赖注入a->去三级缓存拿a的代理
>
> ->把代理的A放入二级->返回a的代理注入到B->b初始化->走后置通知获得代理b->B的代理放入一级缓存、
>
> ->原生a依赖注入b->A初始化->后置通知，不走代理返回原生A->再偷天换日->放入一级缓存

10.Spring Aop的理解和使用?

Aop面向切面编程，在spring中，通过配置**切面类和切点，切面类中主要有前置通知方法，后置通知方法，异常通知方法以及环绕通知方法**，切面类中的方法的逻辑就是你要织入代码的逻辑，然后你需要判断哪些类需要进行前面切面，所以就要配置切入点，可以使用xml方式，也可以使用注解的方法。

实现方式主要是通过动态代理给目标类生成代理对象，如果目标类实现了接口就使用jdk代理，如果没有实现接口使用cglib代理，如果配置了优先级，则默认使用cglib代理。在spring容器启动后，就会去扫描哪些需要进行aop的类，这些类有哪些切面（切面是可以配置多个的），然后给目标类的每个方法都绑定一个相应的执行链，并放入到一个map中，当你调用时，会走代理类，然后通过**索引加递归**的方式去调用切面类前置通知方法，目标类方法，后置通知方法这样的顺序去调用。

使用：可以拦截目标类，在方法的前后织入日志打印逻辑，包括spring事务开启也是使用的aop,还有动态数据源切换等场景都是可以使用aop。如果配置多个切面，则要指定执行顺序，通过设置**sort的大小来指定，sort值越小，越先执行，越后结束，sort值越大，越后执行，越先结束**，就相当于一个同心圆。事务sort值默认是int类型的最大值。

11.spring中的AOP是在DI之前还是之后

1.对于单个对象创建，我们可以看到当前对象AOP是在DI之后

2.循环依赖的时候 ，AOP在DI之间

12.对于spring中的bean的懒加载如何实现

咕泡出品 必属精品

@lazy

多例的bean

13.spring中声明式事务注解中的参数

1.propagation传播机制

2.isolation隔离级别

3.timeout超时时间，如果共用一个事务，只有之前的事务设置的timeout有效

4.rollbackFor回滚异常

5.noRollbackFor不回滚异常

14.事务的特点

事务应该具有 4 个属性：原子性、一致性、隔离性、持久性

原子性（Automicity）

一个事务是一个不可分割的工作单位，事务中包括的诸操作要么都做，要么都不做。

一致性（Consistency）

事务必须是使数据库从一个一致性状态变到另一个一致性状态。

一致性与原子性是密切相关的。

隔离性（Isolation）

一个事务的执行不能被其他事务干扰。

即一个事务内部的操作及使用的数据对并发的其他事务是隔离的，并发执行的各个事务之间不能互相干扰。

持久性（Durability）

持久性也称永久性（Permanence），指一个事务一旦提交，它对数据库中数据的改变就应该是永久性的。接下来的其他操作或故障不应该对其有任何影响。

15.spring的事务失效场景

1.非public方法 事务的实现原理是代理增强，非public不能进行代理增强，不能进行JDK或者CGLIB代理

演示： 把methodA改成protect

2.调用本类的方法 （调用methodC方法）

3.抛出捕捉的非RuntimeException，如果想要捕捉所有异常（rollbackFor = {Exception.class}）

或者指定@Transactional(rollbackFor = {RuntimeException.class,ClassNotFoundException.class},propagation = Propagation.REQUIRED) 多个异常类

4.我们知道事务就是依赖于数据库的，所以数据库不支持肯定也是失效的！！ 比如myIsam

16.对spring事务的理解?

咕泡出品 必属精品

spring的事务开启方式有两种，一个是声明式事务，一个是编程式事务，声明式事务是通过添加Transaction注解的方式开启事务，一般加在类或者方法上，事务控制粒度比较大，但使用上比较方便，编程式事务通过TransactionTemplate,控制事务的粒度小，但是代码侵入性比较强。使用spring aop实现。

spring事务提供了7中传播机制：

1. REQUIRED(必须的):是默认的传播机制，如果B方法中调用A,如果B中有事务，则A无论是否开启事务都会用B的事务，任何地方出现异常A和B都回滚
2. REQUIRES_NEW(需要新的)：每次都会开启一个新的事务，外面事务回滚，里面事务不会回滚
3. SUPPORTS(1)： 如果存在一个事务，支持当前事务。如果没有事务，则非事务的执行，完全依赖最外层事务
4. MANDATORY(强制性的)：必须运行在事务里面
5. NOT_SUPPORTED：以非事务方式执行操作，如果当前存在事务，就把当前事务挂起
6. NEVER：以非事务方式执行，如果当前存在事务，则抛出异常
7. NESTED：开启新事务，提交事务依赖于外层事务，如果外层事务回滚，则里面事务也回滚

spring事务还提供了四种隔离级别：

1. DEFAULT(-1):数据库默认的隔离级别
2. READ_UNCOMMITTED(1):读未提交 ru,会导致脏读
3. READ_COMMITTED(2):读已提交 rc 避免脏读，允许不可重复读和幻读
4. REPEATABLE_READ(4):可重复读 rr 避免脏读，不可重复读，允许幻读,innodb存储引擎解决了幻读
5. SERIALIZABLE：串行化

从上到下，隔离级别越来越高，并发性能就越来越差，spring事务的本质还是数据库的事务，如果数据库不支持事务，spring的事务也就没有了意义

17.springmvc的流程?

1.用户发送请求至前端控制器DispatcherServlet；

2.DispatcherServlet收到请求后，调用HandlerMapping处理器映射器，请求获取Handler；

3.处理器映射器根据请求url找到具体的处理器Handler，生成处理器对象及处理器拦截器(如果有则生成)，一并返回给DispatcherServlet；

4.DispatcherServlet 调用 HandlerAdapter处理器适配器，请求执行Handler；

5.HandlerAdapter 经过适配调用 具体处理器进行处理业务逻辑；

6.Handler执行完成返回ModelAndView；

7.HandlerAdapter将Handler执行结果ModelAndView返回给DispatcherServlet；

8.DispatcherServlet将ModelAndView传给ViewResolver视图解析器进行解析；

9.ViewResolver解析后返回具体View；

10.DispatcherServlet对View进行渲染视图（即将模型数据填充至视图中）

11.DispatcherServlet响应用户。

18.Spring事务和Mysql事务有什么区别?

Spring事务本质是对数据库事务的支持，如果数据库不支持事务（例如MySQL的MyISAM引擎不支持事务），则Spring事务也不会生效。

咕泡出品 必属精品

19.加了Spring的事务注解后，用try catch捕获异常有什么影响（面试官问的）

如果没有设置捕获的异常，然后捕获的是RuntimeException异常，则会回滚，如果不是，不会回滚

可以自己设置回滚的异常，也可以设置不回滚的异常

20.Spring如何保证线程安全的？（面试官问的）

这个问题主要是针对Spring中的bean的线程安全问题。

Spring中的Bean默认是单例的，单例的是被所有线程共享的，所以会存在安全问题，如果想要线程安全，可以设置为多例的Bean。

追问：那么为什么有些bean是单例的呢，却又是安全的？

因为一般来说线程不安全是说共享的资源不安全，但是有些bean其实没又资源的概念，比如dao接口，只有想实体类，我们一般设置为多例的，因为里面包含了数据资源。

21.Spring Bean各作用域之间的区别？

1.singleton:这种bean范围是默认的，这种范围确保不管接受到多个请求，每个容器中有一个bean的实例，单利模式由bean factory自身来维护。

2.Protype:原先通过范围与单例范围相反，为每一个bean请求提供一个实例。

3.request：在请求bean范围内会每一个来自客户端的网络请求创建一个实例，在请求完成之后，bean会失效并被垃圾回收器回收。

4.session：与请求范围类似，确保每个session中的bean的实例在session过期后bean会随之消失。

5.global-session:global-session和portlet公用全局存储变量的话，那么这全局变量需要存储在global-session中。

22.Springmvc中是拦截器先运行还是过滤器先运行（面试官问的）

> 过滤器先运行

mybatis面试题

Mybatis的整体流程能简单讲下吗？

- 初始化工作

> 1.解析配置文件，得到sqlSessionFactory,把配置文件中的参数都保存到configuration类中
>
> 2.解析mapper.xml文件，把每一个语句封装成一个MappedStatement，并且放入到Map；key:namespace+sqlid value:MappedStatement
>
> 3.为每一个mapper接口类都会封装一个MapperProxyFactory与namespace映射关系保存到map中

- 会话创建

> 创建会话去执行sql，肯定离不开SqlSession，我们先介绍下什么是SqlSession,就像在Navicat中打开的一个窗口，多个窗口就是多个sqlSession

咕泡出品 必属精品

> 1. 创建事务
> 2. 创建执行器，分别有三种执行器
> 3. 返回DefaultSqlSession对象

- 语句执行

> 1. 从knownMappers获取该接口的代理工厂类
> 2. 使用JDK动态代理创建接口的代理类,MapperProxy 为代理类，用于代理Mapper接口方法，调用mapper接口的方法都会到代理类MapperProxy的invoke方法
> 3. 获取MappedStatement，这个在初始化的时候，我们就知道存的是什么，一个sql语句标签对应一个MappedStatement
> 4. 通过执行器执行语句，返回结果

为什么调用dao接口的方法就能调用sql查询数据，实现原理是什么?

- 在第一步build的时候就会根据mapper XML里面的nameSpace拿到接口并且创建一个代理类工厂new MapperProxyFactory<>(type)，存入我们的knownMappers集合，此时还没有创建代理类：入口在parse的bindMapperForNamespace(); 同时会把sql语句放入mappedStatements的一个map中
- 在调用session.getMapper(BlogMapper.class)方法时，会根据class创建一个代理类，并且返回代理类
- 执行方法时，执行的时我们代理类的方法MapperProxy，在代理类方法中，执行我们的逻辑，并且从MappedStatement拿到一开始就加载好的sql语句

mybatis的一级缓存和二级缓存你了解吗？能说下他们的区别吗?

- 一级缓存：作用域为SqlSession，存在BaseExecutor下的PerpetualCache下的cache，默认开启。存入在查询的方法里面
- 二级缓存：作用域为nameSpace
  - 二级缓存开启需要2个条件：

    > 1.要有CachingExecutor，所以由cacheEnabled配置决定，默认是true
    >
    > 2.Mapper必须要有Cache标签，也就是决定currentCache有没有
    >
    > 3.还要sql上useCache标签为true,对于query语句默认为true,如果想关闭，可以设置为false

追问：什么时候有清除?

- 增删改查的标签上有属性：flushCache="true"，select语句默认是false，一级二级缓存都会被清理
- 如果开启了事务，则最后都是在commit或者rollback后进行处理
- 修改和删除都会清除缓存

Mybatis中的执行器你了解吗？他们的区是什么?

simple: 普通执行器，默认， 对每条sql进行预编译->设置参数->执行等操作

reuse:执行器会重用预处理语句（PreparedStatement）

batch:执行器不仅重用语句还会执行批量更新 批量执行器, 对相同sql进行一次预编译, 然后设置参数, 最后统一执行操作

咕泡出品 必属精品

Mybatis中的xml映射文件中的sql的id是否可以重复

如果同一个nameSpace不可以,会报错

如果不是同一个nameSpace不可以

Mybatis是如何进行分页的？分页插件的原理是什么？

Mybatis 使用 RowBounds 对象进行分页，它是针对 ResultSet 结果集执行的内存分页，而非物理分页。

可以在 sql内直接书写带有物理分页的参数来完成物理分页功能，也可以使用分页插件来完成物理分页，分页插件的基本原理是使用 Mybatis 提供的插件口，实现自定义插件，在插件的拦截方法内拦截待执行的 sql，然后重写 sql

Mybatis四大拦截对象顺序是怎样的？

mybatis提供了四大拦截对象，分别为：

拦截Exectutor

拦截sql语法构建的处理 StatementHandle

拦截参数 ParameterHandler

拦截结果集 ResultSetHandler

#{}和${}的区别

{}是预编译处理，${}是字符串替换。 Mybatis 在处理#{}时，会将 sql 中的#{}替换为?号，调用 PreparedStatement 的 set 方法来赋值； Mybatis 在处理时，就是把 {}替换成变量的值。 使用#{}可以有效的防止 SQL注入，提高系统安全性。

mybatis和spring如何集成的

> mybatis的流程是初始化，回话创建，语句执行，那么mybatis和spring集成也需要这样
>
> 初始化：
>
> 1.要的到一个SqlSessionFactory,所以通过实现Spring中的FactoryBean去调用getObject()方法的到SqlSessionFactory
>
> 2.原来这个步骤是再初始化中做，那么很显然getObject()完成了初始化
>
> 3.为了更好的利用bean的生命周期，实现了InitializingBean接口，再getObject()方法中调用afterPropertiesSet()方法去完成mybatis的初始化
>
> 4.通过MapperScannerRegistrarNotFoundConfiguration类会去调用AutoConfiguredMapperScannerRegistrar中的registerBeanDefinitions方法放入到spring容器中

> 开启回话：

咕泡出品 必属精品

> 1.但是这个是再启动spring后，调用的时候才开启回话，所以spring集成mybatis的时候弄了一个SqlSessionTemplate的类，去给sqlSession生成代理类，
>
> 2.然后去调用SqlSessionInterceptor类中invoke方法
>
> 3.调用sqlSessionFactory.openSession()得到一个SqlSession类

> 语句执行：
>
> 1.执行方法
>
> 2.是否需要提交
>
> 3.最后关闭Session

Springboot面试题

1.springBoot的启动流程

> 实例化SpringApplication:
>
> 1.推断WebApplicationType，主要思想就是在当前的classpath下搜索特定的类
>
> 2.搜索META-INF\spring.factories文件配置的ApplicationContextInitializer的实现类
>
> 3.搜索META-INF\spring.factories文件配置的ApplicationListenerr的实现类推断MainApplication的Class
>
> run方法:
>
> 1.创建一个StopWatch并执行start方法，这个类主要记录任务的执行时间
>
> 2.配置Headless属性，Headless模式是在缺少显示屏、键盘或者鼠标时候的系统配置
>
> 3.在文件META-INF\spring.factories中获取SpringApplicationRunListener接口的实现类EventPublishingRunListener，主要发布SpringApplicationEvent
>
> 4.把输入参数转成DefaultApplicationArguments类
>
> 5.创建Environment并设置比如环境信息，系统熟悉，输入参数和profile信息
>
> 6.打印Banner信息
>
> 7.创建Application的上下文，根据WebApplicationTyp来创建Context类，如果非web项目则创建AnnotationConfigApplicationContext，在构造方法中初始化AnnotatedBeanDefinitionReader和ClassPathBeanDefinitionScanner
>
> 8.在文件META-INF\spring.factories中获取SpringBootExceptionReporter接口的实现类FailureAnalyzers
>
> 9.准备application的上下文
>
> 1.初始化ApplicationContextInitializer
>
> 2.执行Initializer的contextPrepared方法，发布ApplicationContextInitializedEvent事件
>
> 3.如果延迟加载，在上下文添加处理器LazyInitializationBeanFactoryPostProcessor

咕泡出品 必属精品

> 4.执行加载方法，BeanDefinitionLoader.load方法，主要初始化了AnnotatedGenericBeanDefinition
>
> 5.执行Initializer的contextLoaded方法，发布ApplicationContextInitializedEvent事件
>
> 10.刷新上下文（后文会单独分析refresh方法），在这里真正加载bean到容器中。如果是web容器，会在onRefresh方法中创建一个Server并启动

3.springBoot的自动装配

> 1.步骤一：
>
> Spring Boot自动配置的核心注解是@SpringBootApplication，该注解是spring boot的启动类注解，它是一个复合注解，里面包含：
>
> @SpringBootConfiguration：该注解上有一个 @Configuration注解，表示这个spring boot启动类是一个配置类，最终要被注入到spring容器中。
>
> @EnableAutoConfiguration：表示开启自动配置，它也是一个复合注解，里面包含：
>
> @AutoConfigurationPackage，该注解上有一个@Import(AutoConfigurationPackages.Registrar.class)注解，其中 Registrar 类的作用是将启动类所在包下的所有子包的组件扫描注入到spring容器中。
>
> @Import(AutoConfigurationImportSelector.class)：其中AutoConfigurationImportSelector类中有一个getCandidateConfigurations()方法，该方法通过SpringFactoriesLoader.loadFactoryNames()方法查找位于META-INF/spring.factories文件中的所有自动配置类，并加载这些类。
>
> 所以spring boot在整个的启动过程中，其实就是在类路径的META-INF/spring.factories 文件中找到EnableAutoConfiguration对应的所有的自动配置类，然后将所有自动配置类加载到spring容器中。
>
> 2.步骤二：
>
> 前面加载的所有自动配置类并不是都生效的，每一个xxxAutoConfiguration自动配置类都是在某些特定的条件之下才会生效。这些条件限制是通过
>
> ```
> @ConditionalOnBean：当容器里存在指定bean的条件下。
> @ConditionalOnMissingBean：当容器里不存在指定bean的条件下。
> @ConditionalOnClass：当类路径下有指定类的条件下。
> @ConditionalOnMissingClass：当类路径下不存在指定类的条件下。
> @ConditionalOnProperty：指定的属性是否有指定的值，比如
> @ConditionalOnProperties(prefix="xxx.xxx", value="enable", matchIfMissing=true)，代表当xxx.xxx为enable时条件的布尔值为true，如果没有设置的情况下也为true。
> ```

4.springboot starter组件

> 1.创建两个模块，一个是starter和自动配置模块
>
> 2.我们先将自动配置模块导入starter中，让启动模块依赖自动配置模块
>
> 3.对自动配置类进行自动配置代码编写
>
> @ConfigurationProperties注解获取配置文件属性
>
> @Configuration 添加配置类
>
> 4.把配置类配置到spring.factories文件中，key为org.springframework.boot.autoconfigure.EnableAutoConfiguration=配置类

咕泡出品 必属精品

5.SPI机制

> java spi机制：
>
> META-INF/services/ 目录：实例化该目录下的类 。
>
> 通过ServiceLoader.load方法找到META-INF/services/ 目录下的文件，然后拿到文件配置的全限定名，根据反射去实例化该类
>
> 无论使不使用，都会去实例化
>
> springboot spi机制：
>
> META-INF/spring.factories/ 目录：根据key找到相应的类去实例化
>
> SpringFactoriesLoader加载会加载META-INF/spring.factories
>
> 从CLASSPATH下的每个Jar包中搜寻所有META-INF/spring.factories配置文件
>
> 然后将解析properties文件，找到指定名称的配置后返回，工厂模式反射创建好bean
>
> 需要注意的是，其实这里不仅仅是会去ClassPath路径下查找，会扫描所有路径下的Jar包，只不过这个文件只会在Classpath下的jar包中。
>
> dubbo spi机制：
>
> META-INF/services/ 目录：该目录下的 SPI 配置文件是为了用来兼容 Java SPI 。
>
> META-INF/dubbo/ 目录：该目录存放用户自定义的 SPI 配置文件。
>
> META-INF/dubbo/internal/ 目录：该目录存放 Dubbo 内部使用的 SPI 配置文件。

咕泡出品 必属精品